Reelwright — YouTube upload implementation

Reelwright — YouTube upload implementation and live evidence

Project: Reelwright (reelwright-513015), project number 984445733063
OAuth client: Desktop app, PKCE, loopback redirect
http://127.0.0.1:8877/api/youtube/oauth/callback
Scope requested: https://www.googleapis.com/auth/youtube.upload (only scope used)
Endpoint called: POST https://www.googleapis.com/upload/youtube/v3/videos?
uploadType=resumable&part=snippet,status

Live run on 28 October 2026, channel "Average Cow":
  request : POST /api/jobs/1a03c4f8-409f-42f5-b582-ced4958bedf6/publish {"privacy":"private"}
  result :
{"state":"done","progress":100,"videoId":"SD4PneZtVh4","url":"https://youtu.be/SD4PneZtVh4"}
  privacy : private — YouTube forces private for API clients that have not completed this audit.

The upload is a resumable session: open the session, then PUT 8 MiB chunks with
Content-Range, then read the final video resource. A dropped chunk is recovered by
asking YouTube how many bytes it has (308 + Range) and resuming from that offset.
The source below is the complete module that performs it.

SERVER/SRC/YOUTUBE/UPLOAD.TS

```
import { open } from "node:fs/promises";
import type { YouTubePrivacy } from "@reelwright/shared";
import { TIMEOUTS } from "../timeouts.ts";
import { providerErrorText, withTimeout } from "../util/http.ts";
import type { HttpRequest } from "../pipeline/tts/openai.ts";

export const YOUTUBE_UPLOAD_URL = "https://www.googleapis.com/upload/youtube/v3/videos?
uploadType=resumable&part=snippet,status";
/** Chunks must be multiples of 256 KiB (except the last). */
export const CHUNK_QUANTUM = 256 * 1024;
export const DEFAULT_CHUNK = 32 * CHUNK_QUANTUM; // 8 MiB

export interface VideoMeta {
  title: string;
  description: string;
  tags: string[];
  privacy: YouTubePrivacy;
}

/** YouTube rejects `<` and `>` in titles and descriptions and caps them at 100 / 5000
characters. */
export function cleanMeta(m: VideoMeta): VideoMeta {
  const strip = (s: string) => s.replace(/[<>]/g, "").replace(/\s+\n/g, "\n").trim();
  const title = strip(m.title).replace(/\s+/g, " ").slice(0, 100) || "Untitled";
  return { ...m, title, description: strip(m.description).slice(0, 5000), tags:
m.tags.map((t) => strip(t)).filter(Boolean).slice(0, 30) };
}

/** Step 1: open a resumable session. The answer's Location header is the session URI. */
export function buildInitRequest(accessToken: string, bytes: number, meta: VideoMeta):
HttpRequest {
  const m = cleanMeta(meta);
  return {
```

```
    url: YOUTUBE_UPLOAD_URL,
    method: "POST",
    headers: {
      Authorization: `Bearer ${accessToken}`,
      "Content-Type": "application/json; charset=UTF-8",
      "X-Upload-Content-Length": String(bytes),
      "X-Upload-Content-Type": "video/mp4",
    },
    body: JSON.stringify({
      snippet: { title: m.title, description: m.description, tags: m.tags, categoryId: "22"
},
      status: { privacyStatus: m.privacy, selfDeclaredMadeForKids: false },
    }),
  };
}

/** Headers for one chunk [start, end] (inclusive) of a `total`-byte file. */
export function chunkHeaders(accessToken: string, start: number, end: number, total: number):
Record<string, string> {
  return {
    Authorization: `Bearer ${accessToken}`,
    "Content-Length": String(end - start + 1),
    "Content-Range": `bytes ${start}-${end}/${total}`,
    "Content-Type": "video/mp4",
  };
}

/** "How much did you get?" — an empty PUT with `Content-Range: bytes * /total`, answered
with 308 + Range. */
export function statusQueryHeaders(accessToken: string, total: number): Record<string,
string> {
  return { Authorization: `Bearer ${accessToken}`, "Content-Length": "0", "Content-Range":
`bytes */${total}` };
}

/** Next byte offset from a 308's Range header (`bytes=0-524287` → 524288; absent → 0). */
export function nextOffset(range: string | null): number {
  if (!range) return 0;
  const m = /^bytes=0-(\d+)$/.exec(range.trim());
  if (!m?.[1]) throw new Error(`unexpected Range header from YouTube: ${range}`);
  return Number(m[1]) + 1;
}

export interface UploadDeps {
  fetchImpl: typeof fetch;
  /** Current access token; `force` asks for a refresh (after a 401). */
  accessToken: (force: boolean) => Promise<string>;
  signal?: AbortSignal;
  onProgress?: (sent: number, total: number) => void;
  chunkSize?: number;
  maxRetries?: number;
  /** Injected for tests; production waits for real. */
  sleep?: (ms: number) => Promise<void>;
}

const realSleep = (ms: number) => new Promise<void>((r) => setTimeout(r, ms));

class Retryable extends Error {}

/** Dropped connections (fetch TypeError), timeouts, 5xx and expired tokens are worth
retrying. */
```

```
function isTransient(err: unknown): boolean {
  return err instanceof Retryable || err instanceof TypeError || (err instanceof Error &&
err.name === "TimeoutError");
}

/**
 * Upload a file with the resumable protocol: init → chunks; after a dropped connection or a
5xx,
 * ask the session how many bytes it holds and continue from there. A 401 refreshes the token
once.
 * Returns the new video id.
 */
export async function resumableUpload(file: string, total: number, meta: VideoMeta, deps:
UploadDeps): Promise<string> {
  const chunk = deps.chunkSize ?? DEFAULT_CHUNK;
  if (chunk % CHUNK_QUANTUM !== 0) throw new Error(`chunk size must be a multiple of
${CHUNK_QUANTUM}`);
  const maxRetries = deps.maxRetries ?? 5;
  const sleep = deps.sleep ?? realSleep;
  const call = (url: string, init: RequestInit) => deps.fetchImpl(url, { ...init, signal:
withTimeout(deps.signal, TIMEOUTS.youtubeRequestMs) });

  // 1. open the session (refresh once on 401)
  let token = await deps.accessToken(false);
  let init = buildInitRequest(token, total, meta);
  let res = await call(init.url, { method: init.method, headers: init.headers, body:
init.body ?? null });
  if (res.status === 401) {
    token = await deps.accessToken(true);
    init = buildInitRequest(token, total, meta);
    res = await call(init.url, { method: init.method, headers: init.headers, body: init.body
?? null });
  }
  if (!res.ok) throw new Error(`YouTube upload init: ${providerErrorText(res.status, await
res.text())}`);
  const session = res.headers.get("location");
  if (!session) throw new Error("YouTube did not return an upload session URI");

  const fh = await open(file, "r");
  try {
    let offset = 0;
    let retries = 0;
    const settle = async (r: Response): Promise<string | null> => {
      if (r.status === 200 || r.status === 201) {
        const body = (await r.json()) as { id?: unknown };
        if (typeof body.id !== "string") throw new Error("YouTube finished the upload but
returned no video id");
        return body.id;
      }
      if (r.status === 308) {
        const next = nextOffset(r.headers.get("range"));
        // Only real forward progress refills the retry budget; a status query alone must
not.
        if (next > offset) {
          retries = 0;
          deps.onProgress?.(next, total);
        }
        offset = next;
        return null;
      }
      if (r.status === 401) {
```

```
        token = await deps.accessToken(true);
        throw new Retryable("access token expired");
      }
      if (r.status >= 500) throw new Retryable(`YouTube HTTP ${r.status}`);
      // 404/410: the session expired; any other 4xx is a final answer from YouTube.
      throw new Error(`YouTube upload: ${providerErrorText(r.status, await r.text())}`);
    };

    for (;;) {
      try {
        const end = Math.min(offset + chunk, total) - 1;
        const buf = Buffer.alloc(end - offset + 1);
        await fh.read(buf, 0, buf.length, offset);
        const id = await settle(await call(session, { method: "PUT", headers:
chunkHeaders(token, offset, end, total), body: buf }));
        if (id) {
          deps.onProgress?.(total, total);
          return id;
        }
      } catch (err) {
        if (deps.signal?.aborted) throw err;
        if (!isTransient(err) || retries >= maxRetries) throw err instanceof Retryable ? new
Error(`YouTube upload gave up after ${retries} retries: ${err.message}`) : err;
        retries++;
        await sleep(TIMEOUTS.youtubeRetryBaseMs * 2 ** (retries - 1));
        // Ask the session what it already has, then continue from there. If that query also
fails
        // transiently, the next turn re-sends from the last confirmed offset (still within
maxRetries).
        try {
          const id = await settle(await call(session, { method: "PUT", headers:
statusQueryHeaders(token, total) }));
          if (id) return id;
        } catch (qerr) {
          if (deps.signal?.aborted || !isTransient(qerr)) throw qerr;
        }
      }
    }
  } finally {
    await fh.close();
  }
}
```